

TRAILFORGE

Technical Reference and User Guide

A durable execution harness for bounded agent workflows, evidence-backed checks and host-controlled decisions.

Document identity	Value
Package / release	trailforge / v0.1.0a1 (public alpha)
Maintainer / license	codera647 / MIT
Document date	8 October 2026
Source authority	Released implementation, tests and checked release evidence
Audience	Application engineers, platform engineers, reviewers and maintainers
Presentation	All text is black; diagrams and tables use monochrome styling

This guide describes the implemented alpha. It distinguishes a workflow completing its checks from a model producing a correct answer, and a recorded approval from a host enforcing permission at an external write.

Repository: <https://github.com/codera647/Trailforge>

Release: <https://github.com/codera647/Trailforge/releases/tag/v0.1.0a1>

Contents

The chapter entries are clickable. Use chapters 4-5 for a first run, 6-19 to embed the harness, and 20-25 for adoption and operations. The API appendix lists signatures extracted from the released source.

01 Purpose, scope and release boundaries	3
02 Thinking model and agentic design patterns	4
03 Architecture and module map	5
04 Installation and first-run setup	6
05 CLI walkthrough and command reference	8
06 Goal, Task and adapter contracts	9
07 Runtime lifecycle and completion semantics	10
08 Budgets, leases and uncertain outcomes	11
09 Persistence, integrity and protocol evolution	13
10 Offline Review profile and exact evidence	14
11 Develop profile and protected acceptance	15
12 Linux Docker sandbox and runner receipts	16
13 Artifacts and revision-bound retrieval	18
14 Publication approvals and reconciliation	19
15 Legacy Review PostgreSQL authority and dispatch	21
16 Broker roles, proposals and schema catalog	23
17 LangGraph facade and event telemetry	24
18 Worked example: a resumable data pipeline	26
19 Worked example: candidate creation and handoff	28
20 Embedding models, tools and product profiles	29
21 Security model and threat boundaries	30
22 Operations, recovery and observability runbook	31
23 Testing, conformance and release evidence	32
24 Troubleshooting and safe diagnostic choices	34
25 Adoption checklist, compatibility and roadmap	35
26 API reference extracted from the release	37
27 Glossary and source references	45

01 / Purpose, scope and release boundaries

Trailforge provides a reusable control layer around application-owned work. It records what was admitted, binds work to an explicit scope, limits declared resource use, persists checkpoints and requires named checks before declaring a general goal complete. The host supplies the actual models, tools, user identity and product policy.

What the name represents

A trail is the durable record of a run: inputs, reservations, outputs, receipts and decisions. A forge is the bounded place where a candidate is built and checked. Together, Trailforge means creating useful work while preserving the evidence needed to understand, recover and review it. This is a guide-level explanation of the name, rather than a new protocol guarantee.

Implemented surface	Meaning in this alpha
Neutral Goal/Task runtime	SDK-free Python adapters; SQLite authority; typed verification; bounded attempts and shared declared-unit ledgers.
Develop profile	PLAN, RESEARCH, BUILD, DEBUG, VERIFY and HEALTH over a scoped candidate workspace and protected checks.
Offline Review profile	Fixed local fixtures, mock review proposals, exact evidence validation, coverage accounting and draft-only output.
Publication boundary	Bound intents, expiring approvals, durable operation identity and reconciliation; a local simulator is supplied.
Optional extensions	Legacy Review PostgreSQL authority, dispatch and broker; LangGraph scheduling facade; Linux Docker runner.
Host responsibilities	Identity, real provider adapters, external publishers, protected criteria, orchestration, billing, secrets and deployment.

Read the alpha accurately

The repository and downloadable artifacts are public. This does not make the API stable or the package production-certified. The release does not include a production GitHub publisher, a customer OAuth application, a built-in live model provider, a generic PostgreSQL backend for Goal/Task, or an incident-triage profile. The standalone CLI is an offline demonstration, not the complete Diffwise product.

The MIT license covers the original source, documentation and examples. Optional and transitive dependencies retain their own licenses. No PyPI publication is claimed here: install from the tagged source or the release wheel. Pin the alpha deliberately and re-evaluate compatibility before upgrading.

Document authority and vocabulary

The source at tag v0.1.0a1 is the reference for behavior. Some historical repository notes describe work as pending even where release evidence now exists; this guide uses the implemented behavior and actual release results. A receipt is a deterministic integrity record, not a digital signature. A check is a host-defined acceptance predicate. An approval is authority supplied through a trusted host path, not a sentence generated by a model.

02 / Thinking model and agentic design patterns

The control loop is admit, execute, observe, check and persist. The application decides the objective and permitted work before the harness executes adapters. After a checkpoint, the host can resume, inspect uncertainty or request a human decision. The runtime never treats additional reasoning text as permission to expand its scope.

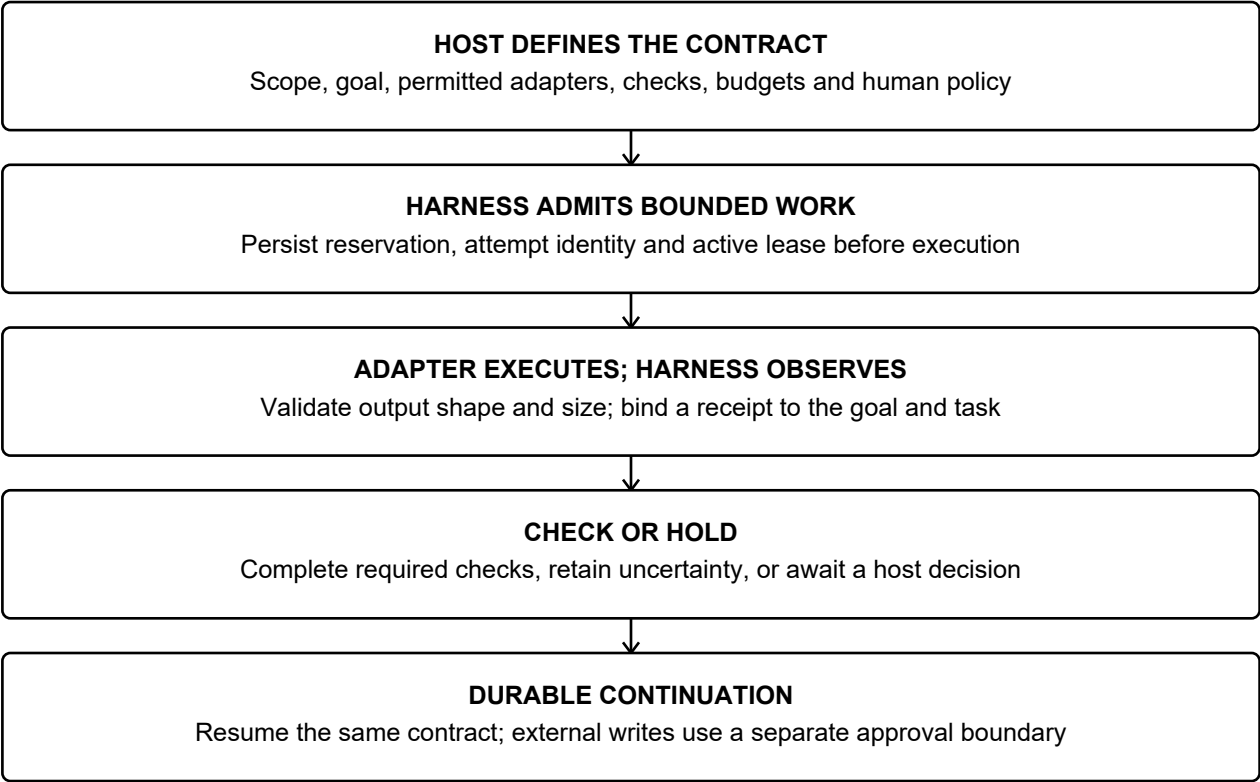


Figure 1. A bounded execution loop; an application may add milestones without changing kernel authority.

Pattern	Concrete Trailforge realization
Planning and decomposition	The host constructs a finite, topologically ordered Task tuple. Develop provides a fixed phase plan.
Tool use	Typed capabilities and TaskRequest isolate orchestration metadata from application callbacks.
Memory and continuity	Durable checkpoint plus journal; the host selects context for a new invocation. No automatic vector memory.
Reflection and verification	Required VERIFY tasks return PASS, FAIL or UNKNOWN. HEALTH checks continuity of the verified candidate.
Human-in-the-loop	UNKNOWN work requires explicit host resolution; external publication requires a bound approval.
Recovery	Compare-and-swap checkpoints, leases, retained reservations and inspect-only reconciliation.
Guardrails	Scope binding, closed contracts, byte bounds, effect policy, protected checks and isolated execution when configured.

Where reasoning belongs

A model may suggest a diagnosis, a plan, a source lookup or a candidate patch. A trusted application validates and converts that suggestion into an allowed action. The model must not choose its own tenant, invent a verifier result, grant a publication approval or reset consumed budgets. Retrieved source remains untrusted data even when it carries a valid integrity receipt.

There is no hidden open-ended self-improvement loop. General tasks are finite and ordered. Develop allows one to three explicitly planned rounds. The Review demonstration has a fixed maximum of two admitted mock calls. Additional work means a new reviewed contract or an explicit permitted recovery decision, rather than an implicit unlimited retry.

Human checkpoints in an application

Useful host checkpoints include approving a changed objective, resolving an uncertain provider outcome, accepting revised test criteria, reviewing a proposed external comment and authorizing deployment. Trailforge provides the durable primitives; it does not supply a universal project-management UI or automatically ask a person after every phase. Choose checkpoints according to actual authority and uncertainty.

03 / Architecture and module map

There are two execution paths. The general Harness runs application-defined Goal/Task contracts. OfflineController implements the fixed Review demonstration. Develop is a profile over the general Harness. They share integrity and persistence concepts, but their checkpoint protocols and state names differ.

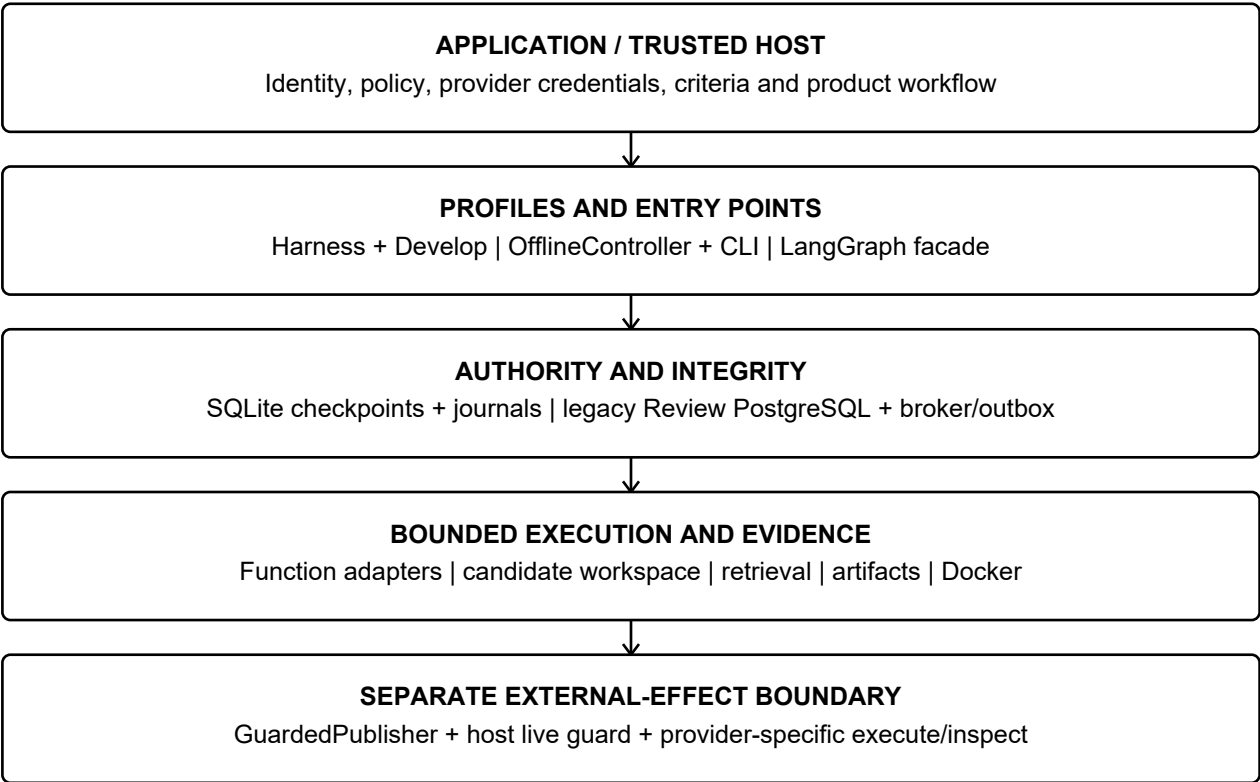


Figure 2. Logical layering. Publisher authorization is separate from neutral task execution.

Module	Role
runtime.py / ports.py	Goal, Task, Harness; adapter capabilities, requests, verified results and declared ports.

Module	Role
develop.py	CandidateWorkspace, fixed Develop phases, continuity and handoff.
controller.py / evidence.py	Offline Review progression, snapshot receipts, proposal validation and coverage.
store.py / core.py	SQLite checkpoints, CAS and journal; canonical JSON, digests and shared contract validation.
publication.py	Intent, GuardedPublisher and durable local simulation.
artifacts.py / adapters/retrieval.py	Scoped byte artifacts and revision-bound literal retrieval.
adapters/sandbox.py	Digest-pinned Linux Docker execution and bounded result receipts.
adapters/postgres.py / dispatch.py	Legacy Review database authority and bounded durable wakeups.
broker.py / adapters/broker.py	Closed action proposals, role/session authority and review simulation requests.
telemetry.py / adapters/langgraph.py	Allowlisted event sink; optional scheduler with kernel-owned state.
cli.py / fixtures.py / examples.py	Offline commands, bounded fixture loading and bundled demonstrations.

Authority versus integration

A scheduling graph, queue notification, adapter reply or UI status is an observation or trigger. The stored contract and the trusted host decide whether that observation can affect a run. Queue consumers reload current scoped authority. A publication adapter checks the provider-side boundary. A verifier must evaluate the same goal and candidate that were admitted.

Protocol namespaces

The neutral execution protocol is `trailforge/execution/0.1`; adapter metadata uses `trailforge/adapter/0.1`. The fixed Review protocol is `offline/0.1`. Develop and publication identify their own contracts as `trailforge/develop/0.1` and `trailforge/publication/0.1`. Do not deserialize a legacy JSON Schema `GoalContract` as the neutral Python `Goal` merely because both describe goals.

04 / Installation and first-run setup

Use Python 3.11 or newer in a dedicated virtual environment. The public alpha was validated on Linux and Windows with Python 3.11 and 3.13. Base installation has no runtime third-party dependencies. Extras add only the optional integrations you select.

Install the tagged source: Linux or macOS shell

```
git clone --branch v0.1.0a1 --depth 1 https://github.com/codera647/Trailforge.git
cd Trailforge
python3 -m venv .venv
. .venv/bin/activate
python -m pip install .
python -m trailforge --version
```

Install the tagged source: Windows PowerShell

```
git clone --branch v0.1.0a1 --depth 1 https://github.com/codera647/Trailforge.git
Set-Location Trailforge
py -3.13 -m venv .venv
.\.venv\Scripts\python.exe -m pip install .
.\.venv\Scripts\python.exe -m trailforge --version
```

The Windows commands invoke the environment interpreter directly, so shell activation and execution-policy changes are unnecessary. If you use a different installed supported interpreter, select it explicitly. The expected package version is 0.1.0a1.

Install the downloaded wheel instead

```
python -m pip install ./trailforge-0.1.0a1-py3-none-any.whl
python -m trailforge --version
```

Download the wheel and SHA256SUMS.txt from the public release page. Verify the wheel against the published digest before installation. The base wheel includes fixtures and packaged schemas/migrations. The standalone Python examples live in the tagged repository and source distribution; they are not bare example-script paths in a wheel-only installation.

Optional extras from the source checkout

```
python -m pip install ".[schemas]"
python -m pip install ".[postgres]"
python -m pip install ".[langgraph]"
```

Extra	Direct pins and purpose
schemas	jsonschema==4.26.0; validation of the legacy closed contract catalog.
postgres	psycopg[binary]==3.3.6 and jsonschema==4.26.0; legacy Review database authority.
langgraph	langgraph==1.2.14; a scheduler facade around the neutral Harness.

Docker is a separate host requirement. Windows local development does not make the native Windows process a Linux isolation boundary. The Linux runner requires a local daemon in Linux-container mode, an already available pinned image and valid workspaces.

Keep local authority private

Choose writable directories for stores, artifacts and simulation files. Exclude those directories, exported source-bearing drafts and credentials from version control. The library does not configure database encryption, secret management or retention policies. Install only the extras you actually use, and record the resolved dependency versions in your application build.

05 / CLI walkthrough and command reference

The CLI runs the fixed offline Review path. It uses local mock outputs, SQLite and local publication simulation. It never contacts a real model provider or posts a GitHub review. The commands below assume a shell using the virtual-environment Python.

Create and run the bundled examples

```
python -m trailforge init trailforge-demo
python -m trailforge run --fixture trailforge-demo/buggy-python \
  --store trailforge.sqlite3 --pause-after snapshot
```

init creates a new directory and refuses to overwrite an existing one. run emits metadata, including the run identifier, to stderr and its JSON result to stdout. Replace RUN_ID below with that actual identifier. Use the same explicit store for every operation.

```
python -m trailforge resume --fixture trailforge-demo/buggy-python \
  --store trailforge.sqlite3 --run-id RUN_ID
python -m trailforge inspect --store trailforge.sqlite3 --run-id RUN_ID
python -m trailforge export --store trailforge.sqlite3 --run-id RUN_ID \
  --output reviewed-draft.json
python -m trailforge evaluate --suite trailforge-demo
```

The multiline examples above use POSIX backslash continuation. In PowerShell, place each command on one line or use PowerShell continuation syntax. The bundled buggy Python fixture demonstrates an empty-input mean failure with exact evidence; a clean fixture demonstrates no validated findings. Both use deterministic supplied proposals.

Command	Important options and behavior
init	Optional destination, default trailforge-demo; destination must not already exist.
run	--fixture required; --store defaults to trailforge.sqlite3; --pause-after snapshot, review or validation.
resume	Same fixture and scope binding; --run-id required; resumes an existing checkpoint.
inspect	--run-id; --store; optional --include-source exposes the full checkpoint.
export	Inspect-style output plus required --output; creates exclusively and refuses overwrite.
evaluate	--suite points to the directory containing suite.json; bounded synthetic cases, temporary store and PASS/FAIL report.
approve	--actor and --simulate required; optional --lose-response; local simulation only.
reconcile	--operation-id and --simulate required; read-only provider inspection in the simulator.

Simulate a lost response and reconcile

```
python -m trailforge approve --store trailforge.sqlite3 --run-id RUN_ID \
  --actor local-reviewer --simulate --lose-response
python -m trailforge reconcile --store trailforge.sqlite3 --run-id RUN_ID \
  --operation-id OPERATION_ID --simulate
```

Use the operation identifier returned by the first command. The simulated provider persists acceptance before losing the reply. The operation can initially be UNKNOWN; reconciliation inspects that durable acceptance and can report ACCEPTED without another send. Only a COMPLETE draft can be approved by this CLI.

Automation and privacy

Exit code 0 is successful command execution, not proof that every product check passed. A PARTIAL review or an UNKNOWN simulation observation can still return 0. Evaluation with status FAIL returns 1; local contract/OS/SQLite denial returns sanitized LOCAL_OPERATION_DENIED with exit code 2. Argument parsing errors also normally return 2. Inspect JSON outcomes as well as exit codes.

Default inspection omits raw snapshot, review and validation internals, but the draft still contains findings and source quotations. Treat it as potentially sensitive. `--include-source` increases exposure and should not be enabled for routine logs.

06 / Goal, Task and adapter contracts

The general runtime is the reusable API for non-PR applications. A Goal declares an immutable scope, objective, finite tasks, named verification requirements, permitted effects and a budget. The host registers trusted adapters before starting the run.

Contract	Fields and defaults
Task	task_id, adapter_id, input; dependencies=(). Dependencies name prior tasks.
Goal	tenant_id, resource_id, objective, tasks, required_checks; allowed_effects=(PURE, SCOPED_READ); budget_units=32; max_task_attempts=3.
AdapterCapabilities	adapter_id, kind; effect=PURE; protocol=trailforge/adapter/0.1; max_input_bytes=65536; max_output_bytes=65536; reservation_units=1.
TaskRequest	run_id, task_id, scope, goal_digest, detached input and completed dependency outputs.
VerifiedResult	verdict in PASS/FAIL/UNKNOWN, goal_digest, evidence. Only a typed verifier result satisfies the verification contract.

Admission rules

A Goal must have one to 128 tasks in topological order, unique task identifiers, a nonempty objective and at least one required check. Every dependency must refer to a task already declared. Required check identifiers must name VERIFY-kind tasks. Registered adapter identifiers are unique and their effect must be allowed by the Goal. A verifier may use only PURE or SCOPED_READ.

The neutral runtime allows PURE, SCOPED_READ and explicitly admitted CANDIDATE_WRITE effects. EXTERNAL_WRITE is not an executable neutral task effect. Route publication through the separate guarded boundary. MODEL, TOOL and VERIFY are adapter kinds; they describe the contract role, not a sandbox privilege level.

Contract hashing and ownership

The Goal digest binds the declared plan and input. The adapter digest binds capability metadata. It does not hash Python callback implementation bytes, import contents or every provider configuration value. The host must version adapter behavior and bind material prompt/model/configuration choices into its own task input or adapter identity when continuity depends on them.

Task data is JSON-bounded and detached before callbacks receive it. Canonical encoding rejects nonfinite numbers, duplicate-key JSON when decoding, unsupported Python objects and oversized payloads. This limits interface ambiguity; it does not make the contents trustworthy instructions.

Construct an adapter

```
from trailforge.ports import AdapterCapabilities, FunctionAdapter

def normalize(request):
    return {"normalized": request.input["text"].strip()}

adapter = FunctionAdapter(
    AdapterCapabilities("example.normalize/1", "TOOL", "PURE"),
    normalize,
)
```

FunctionAdapter executes an application callback synchronously in the trusted host process. Its declared PURE effect is metadata, not an OS restriction. A callback can access Python facilities available to the process. Use a separately enforced execution boundary for untrusted code, and enforce provider timeouts in the adapter.

Declared ports are integration contracts

The port layer declares artifact, retrieval, workflow, publication and telemetry interfaces. A declaration is not a complete service implementation. In particular, PublisherPort declares prepare/execute/inspect, whereas the supplied GuardedPublisher directly uses adapter metadata plus execute/inspect; it does not call a universal prepare hook.

07 / Runtime lifecycle and completion semantics

A checkpoint is the authority for progress. Starting creates a READY record with all tasks pending. Advancing admits and executes tasks in the supplied order, persists each valid result and then evaluates the required checks. Dependencies express ordering and context; this alpha does not schedule independent tasks within one run concurrently.

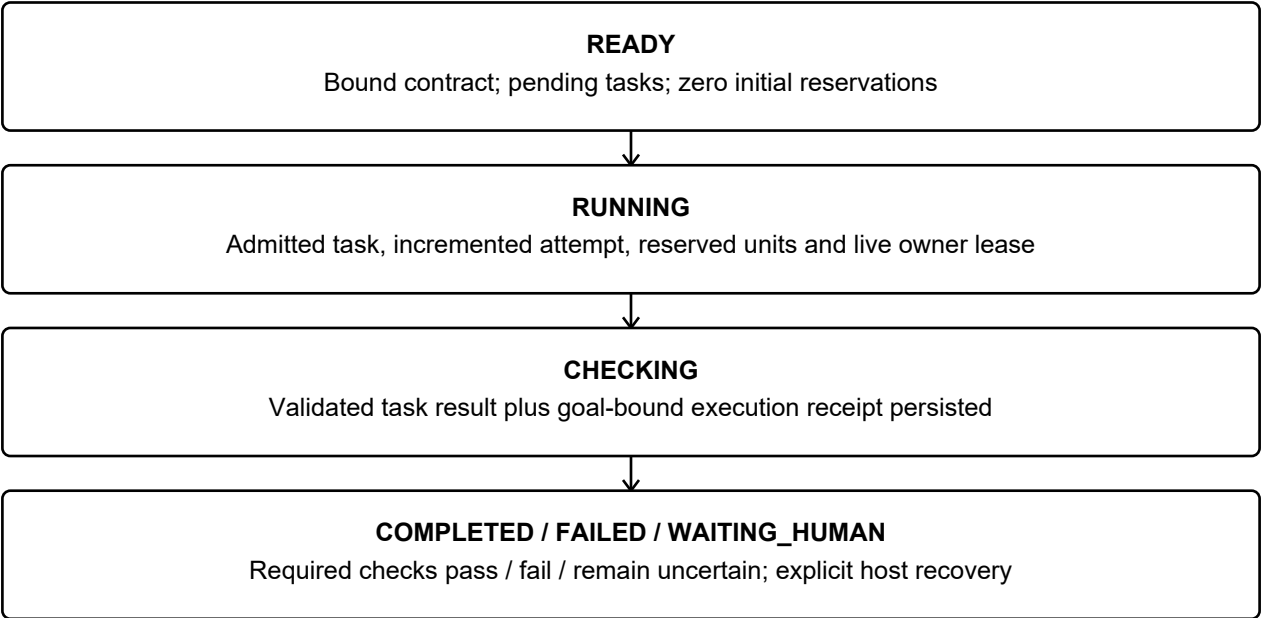


Figure 3. Normal progression. Cancellation and uncertainty can interrupt execution; the persisted record decides continuation.

State or observation	Operational meaning
READY	The run can admit remaining allowed tasks.
RUNNING with live active lease	Another advance conflicts; a second caller cannot replace the live attempt.

State or observation	Operational meaning
CHECKING	A result has been recorded. A requested pause returns here without finishing the goal.
WAITING_HUMAN	Uncertain work, exhausted attempts or a denied budget needs host attention.
COMPLETED / outcome COMPLETE	All required verification tasks passed after the declared task sequence.
FAILED / outcome FAILED	At least one required check returned FAIL at final evaluation.
CANCELLED	No further admitted progress; already incurred work and reservations remain.

Typed checks, not plausible JSON

A VERIFY adapter must return exactly VerifiedResult with a valid verdict and the same Goal digest. An ordinary dictionary that happens to contain a PASS field is not equivalent. The harness binds an output digest, reservation units, attempt number, task identifier and goal digest in its task receipt.

A required FAIL is evaluated after the declared task sequence; it does not automatically stop all following tasks immediately. Design plans so later tasks cannot create dangerous effects based on an unchecked intermediate result. The neutral runtime itself denies external writes, and final publication has its own authorization step.

Resume the same run

Reconstruct the same Goal and adapter capabilities, open the same store and call advance with the original run identifier. Completed task outputs are replayed from the checkpoint rather than re-executed. Scope, goal or adapter metadata changes are denied. Use a new goal/run for a changed contract; do not edit a checkpoint to force compatibility.

Pause points and terminal inspection

pause_after names a task identifier and provides a convenient milestone after its result is recorded. It is not a human approval grant. Terminal advance returns the existing terminal record. General Harness terminal replay does not re-run external application predicates; Develop adds stronger current-workspace handoff checks described later.

08 / Budgets, leases and uncertain outcomes

Trailforge accounts for declared adapter units. Reservation occurs before execution and is persisted with task admission. This makes interrupted and uncertain work visible instead of silently restoring its allowance. These units are not measured tokens, dollars, CPU-seconds or an automatic billing guarantee.

Limit	Actual supported bound
Goal budget_units	1 to 1,000,000 declared units; default 32.
Task attempts	1 to 3 per task; default 3.
Adapter reservation_units	1 to 1,000,000 per admitted invocation.
Input / output byte limits	1 to 1,048,576 each; defaults 65,536 each.
Harness lease	Greater than 0 and at most 300 seconds; default 30.
Tenant / global limits	1 to 1,000,000,000; defaults 1,000 / 10,000; shared only through the same SQLite authority.

Shared accounting

SQLite BEGIN IMMEDIATE protects tenant/global reservation updates together with the checkpoint and journal. Accepted reservations are monotonic: the runtime does not refund them after cancellation, timeout, callback exceptions or UNKNOWN. Reopening the same authority with conflicting configured caps is denied rather than resetting the ledger.

`resource_status()` reports unit caps and reserved totals with `monetary_guarantee=False`. Independent SQLite files do not share a global budget. A host needing organization-wide spend control must provide a single appropriate authority and provider-side accounting, pricing bounds and enforcement.

The three main uncertainty paths

- A callback raises, returns an invalid result or exceeds a contract size bound: the task becomes UNKNOWN and the run waits.
- An active attempt expires before valid completion: the admitted attempt remains consumed and is retired as UNKNOWN.
- A VERIFY adapter returns UNKNOWN: its observation is recorded, but the goal cannot be declared complete.

A synchronous callback may finish after its lease expires. Ownership and expiry prevent its late result from committing, but they do not preempt Python execution or undo a provider request. Adapter-side timeouts and isolated subprocess supervision are necessary where duration or paid work must be bounded.

Explicit host resolution

```
# The trusted host has inspected the uncertain attempt first.
harness.resolve_unknown(run_id, goal, "task-id", disposition="RETRY")
record = harness.advance(run_id, goal)
```

```
# Alternatively, retain the record and stop this goal.
harness.cancel(run_id, goal)
```

These are alternative host actions, not a recommended sequence to run blindly. RETRY clears the task result/receipt and returns it to pending; it retains consumed units and attempt count. CANCEL terminates the run. Retries remain bounded by configured attempts and available units. Even declared PURE work is not automatically reissued after UNKNOWN.

When to involve a person

Use human review when the provider cannot establish whether an effect happened, a changed objective is needed, the remaining budget requires an authorized policy change, or acceptance criteria are ambiguous. A model-generated retry recommendation is input to that decision. It must not call the host resolution authority merely because it asks to continue.

09 / Persistence, integrity and protocol evolution

The neutral runtime and local Review demonstration use SQLiteRunStore. Records carry canonical JSON, checksums, immutable binding identity and a state version. A journal entry is committed with each accepted checkpoint transition. Compare-and-swap prevents stale callers from silently overwriting newer state.

SQLite authority surface	Stored meaning
runs	run_id, binding_digest, version, canonical payload and checksum.
events	Per-run sequence, event kind and state digest; paired with checkpoint writes.
execution_limits	Neutral runtime tenant/global caps and retained reservation totals.
publication_approvals	Bound approval payload/checksum, revocation and consumption state.
publication_operations	Unique exact-intent identity, operation payload and checksum.

What load and save check

Load checks the payload checksum, run identity, expected protocol/binding, state_version against the row and the latest journal sequence/state digest. Save checks the expected version and binding, increments the version and appends its journal event atomically. A stale save raises ConflictError. Multiple protocol records can coexist in one file, but a store instance admits only its configured protocol.

The latest-event consistency check is not a full cryptographic history chain. Neither local checksums nor receipts provide nonrepudiation or protection against an attacker who controls the database and can recompute hashes. Restrict filesystem and database write authority; preserve immutable external release evidence where audit requirements need stronger assurance.

Operational storage rules

- Keep checkpoint, simulator and artifact locations persistent across host restarts. Moving only one file can lose related observations.
- Use a quiescent snapshot or the SQLite backup API for an active store; avoid treating an arbitrary live file copy as a consistent backup.
- Record application release, adapter identities and goal construction inputs so the same contract can be reconstructed.
- Protect stored source, drafts, evidence and approval actor labels with the application retention and access policy.

Evolving a contract

The alpha does not promise schema or API compatibility across future releases. Treat protocol identifiers as real boundaries. When a goal changes, create a new goal/run rather than rewriting an existing digest. For an application data migration, back up the authority, test upgrade/recovery on a copy and retain the prior package version until the new host verifies historical reads.

PostgreSQL migration checksums apply to the separate legacy Review store. There is no universal neutral runtime database migration service, and no automatic migration from the neutral SQLite execution record to PostgreSQL.

10 / Offline Review profile and exact evidence

The fixed Review path is a deterministic demonstration of scope, source integrity, proposal validation and coverage. It does not evaluate a live language model. The four named roles are security, correctness, tests and documentation; the supplied mock adapter returns configured role outcomes and findings in one callback, not four independent agents.

Profile progression

Stage	State and checkpoint purpose
ADMITTED	READY: bind fixture input, revisions, policy and config.
SNAPSHOT	RUNNING: persist captured files with source receipts.
REVIEWED	CHECKING: persist admitted mock invocation and configured proposal output.
VALIDATED	CHECKING: validate evidence, role coverage and bounded findings.
DRAFTED	TERMINAL: persist COMPLETE or PARTIAL draft, mode DRAFT, reviewer MOCK.

ScopeBinding includes tenant/resource, base/head revisions, input digest, policy digest, profile version and configuration digest. Resume checks those bindings and recomputes snapshot/output/validation/draft consistency. Model-call admission is persisted before the mock callback; its fixed policy permits at most two calls. That legacy recovery behavior is distinct from explicit neutral-runtime UNKNOWN resolution.

Fixture bounds and language coverage

A fixture.json manifest contains the profile version, tenant/resource, revisions, file specifications and mock output. It admits one to 32 files, a 65,536-byte manifest, up to 65,536 bytes per file and 262,144 source bytes total. Files must be scoped regular UTF-8 files; canonical relative POSIX paths exclude traversal, backslashes, colons, symlinks and junctions. Loading does not execute repository code.

The fixture suffix classifier explicitly recognizes .py, .js and .ts. Other suffixes are reported as unsupported, including .jsx and .tsx in this fixture loader. Truncated input and role TIMEOUT/FAILED/SKIPPED statuses create coverage gaps. They are not silently counted as reviewed.

Validate a finding against captured bytes

A proposal supplies a title, category, severity, scenario, impact, recommendation, role and evidence. Evidence binds path, one-based start/end line, exact quote, head revision, content digest and snapshot receipt identifier. The validator reconstructs the quoted range with preserved line endings. A changed revision, forged receipt, out-of-range line or normalized quote is denied.

For the bundled mean example, line 3 is exactly `return sum(values) / len(values)` followed by a newline. The proposed scenario is `mean([])`; the stated impact is `ZeroDivisionError` instead of the expected zero. This is a synthetic, known proposal, not proof that a real reviewer would discover it.

Interpret the draft correctly

Validated findings remain marked `needs_human_review=True`. Reliability is unset and the critic is `NOT_IMPLEMENTED`. Deduplication fingerprints category, scenario and evidence. Rejected malformed findings contribute coverage gaps; incomplete role/source coverage produces `PARTIAL`. `COMPLETE` describes workflow coverage under this profile, not semantic correctness, security completeness or calibrated confidence.

The draft digest binds the returned draft. A valid source receipt proves correspondence to captured bytes; it does not prove that the proposed defect is real. Product-specific tests and reviewer acceptance are separate requirements.

11 / Develop profile and protected acceptance

Develop turns an engineering objective into a fixed bounded sequence over an application-owned candidate workspace. It binds a separate protected-checks tree, wraps phase adapters and produces a handoff only after current candidate integrity matches the verified candidate. It is a reusable profile, not an autonomous project generator.

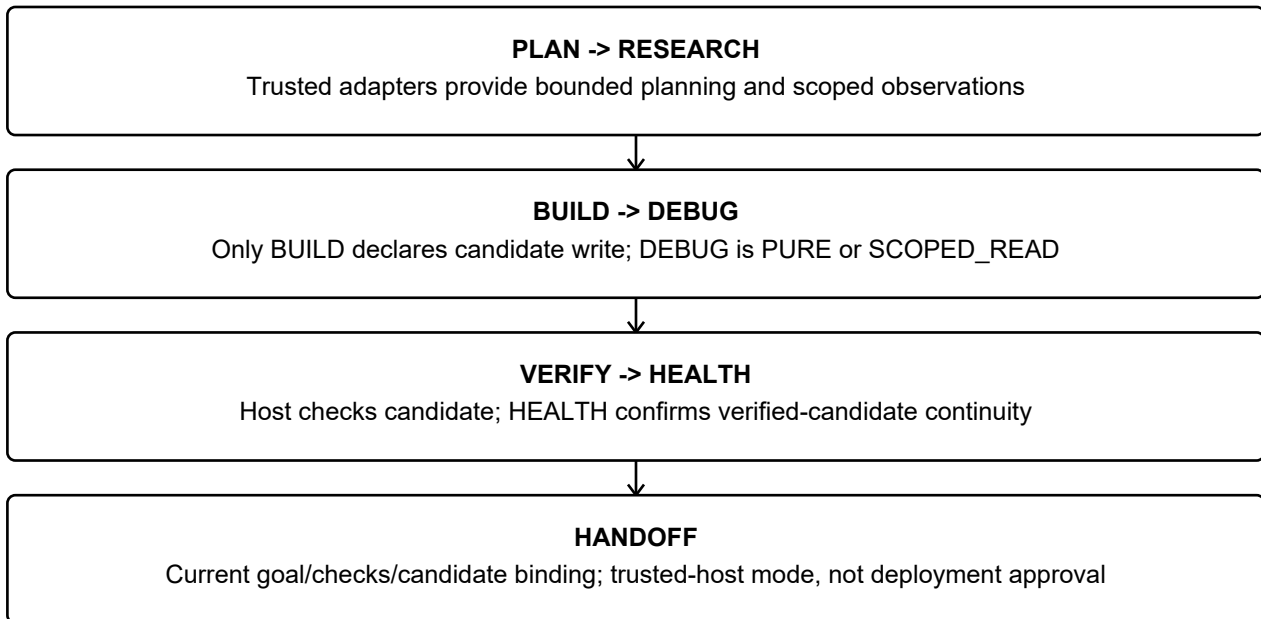


Figure 4. Each explicitly planned round follows these six phases in this order.

Construction rules

Provide adapters for exactly PLAN, RESEARCH, BUILD, DEBUG and VERIFY. HEALTH is supplied by the profile. BUILD must declare CANDIDATE_WRITE. The other supplied phases are PURE or SCOPED_READ. VERIFY must have kind VERIFY and return VerifiedResult. One to three rounds are allowed; the default is one. Task identifiers are 1:PLAN, 1:RESEARCH and so on, with every phase depending on all earlier phases.

Only the last planned round VERIFY and HEALTH tasks are required final checks. Earlier failed verification can be followed by a predeclared later build round. This supports finite correction, not hidden recursive retries. The profile contract binds checks_digest, actual candidate-root identity, allowed paths, per-file byte cap and round count.

CandidateWorkspace

Allowed paths are an explicit tuple of one to 128 canonical relative names, with case-folded duplicates denied. The default maximum file size is 262,144 bytes, configurable up to 1,048,576. Reads and writes enforce those bounds and reject symlink/junction/hardlink paths. Writes use an exclusive temporary file, fsync and atomic replacement.

The checks directory must be separate and nonoverlapping. Phase wrappers compare checks before and after execution and reject a changed criteria tree. A non-BUILD phase that changes the candidate is rejected. These are integrity checks in a trusted host process; they do not replace operating-system privilege isolation against malicious callbacks or concurrent privileged writers.

VERIFY, HEALTH and handoff

VERIFY evidence is wrapped with candidate and checks digests, profile identity and round number. HEALTH passes only when the current candidate matches the candidate verified by that round; otherwise it reports UNKNOWN. `continuity()` gives a compact host checkpoint view, including phase progress and receipt identifiers, without treating model memory as authority.

`handoff()` requires a completed goal, unchanged protected checks and a currently matching candidate. The receipt explicitly says `execution_mode=TRUSTED_HOST_CALLBACKS` and `linux_isolation_verified=False`. Calling Docker inside a custom verifier does not automatically change this handoff claim. A host can attach a separate Docker receipt, but must not relabel the built-in handoff as isolated execution.

A candidate edited after verification invalidates handoff, even when the run previously completed. Handoff is a review artifact; the application still decides whether to commit, merge, publish or deploy.

12 / Linux Docker sandbox and runner receipts

DockerSandbox is the supplied boundary for running a command against read-only candidate and protected-checks trees on a local Linux-container daemon. It validates a digest-pinned local image and returns bounded execution observations. It does not translate exit code zero into semantic acceptance.

Prerequisites and admitted command

The daemon endpoint must be `local unix://` or `npipe://`, report Linux containers and provide the already available image at the exact permitted repository/digest identity. A missing image or failed preflight is denied. The runner never auto-pulls an image or falls back to running the same command in the host shell.

```
--pull never --network none --read-only --cap-drop ALL
--security-opt no-new-privileges --user 65534:65534
--pids-limit 64 --memory 256m --memory-swap 256m --cpus 1
--ulimit nofile=1024:1024
--tmpfs /tmp:rw,noexec,nosuid,size=64m
# Candidate and checks are distinct read-only bind mounts.
# Workdir is /candidate; PYTHONDONTWRITEBYTECODE=1 is supplied.
```

The command arguments are a tuple of one to 64 NUL-free strings with a combined 16,384-byte limit. The runner replaces the image entrypoint with `argv[0]` and passes remaining arguments directly. It does not implicitly invoke a shell. The candidate is not a writable build directory inside this boundary.

SandboxLimits	Default and configurable bound
seconds	30; range 1-300.
memory_mb	256; range 64-4,096.
pids	64; range 16-512.
output_bytes	1,048,576; range 1,024-4,194,304.
tree_digest bounds	512 entries including directories; 16,777,216 total bytes; regular files, no link aliases.

Execution and cleanup

Pre/post tree digests bind the submitted candidate and checks to the result. A bounded reader merges stdout/stderr, and timeout or output-limit supervision stops the owned process/container. Cleanup first verifies the container ownership label and then confirms absence using a healthy daemon result. A daemon error is not accepted as evidence that a container disappeared.

Receipt fields include goal/candidate/checks digests, image, command digest, status, exit_code, output_digest, elapsed seconds, output size and network=NONE. Status may be EXECUTED, TIMEOUT, OUTPUT_LIMIT or WORKSPACE_CHANGED. criterion_verdict remains UNKNOWN; a trusted verifier must interpret the result against protected criteria and return VerifiedResult.

Release-tested image and host boundary

The release tested this exact Python image identity:

```
docker.io/library/python@sha256:f040863673aea2570c3ff6a5c3fb4c673a016cbc5375005ad145915922b6b78a
```

The image digest binds an image snapshot; it does not guarantee the image is free of vulnerabilities. Image refresh is an explicit host change with new validation. The host OS, daemon and Docker administrators remain trusted. Containers are not a defense against a malicious host, and the runner provides no Windows-native untrusted-code sandbox.

Compose a runner in a host verifier

```
from trailforge.adapters.sandbox import DockerSandbox, tree_digest

image = (
    "docker.io/library/python@sha256:"
    "f040863673aea2570c3ff6a5c3fb4c673a016cbc5375005ad145915922b6b78a"
)
runner = DockerSandbox(image)
# Existing separate trees; /checks/check.py is host-controlled criteria.
receipt = runner.run(
    candidate, checks, ("python", "/checks/check.py"),
    goal_digest=request.goal_digest,
    candidate_digest=tree_digest(candidate),
    checks_digest=tree_digest(checks),
)
```

This composition snippet assumes an admitted verifier TaskRequest, existing candidate/checks directories, readable protected check.py and a preloaded image. The host must interpret receipt status, exit_code and bounded output against its criterion. This guide does not rerun Docker conformance; the five-case Linux result is the release snapshot described in chapter 23.

13 / Artifacts and revision-bound retrieval

Evidence and retrieval should be scoped, bounded and reproducible. Trailforge supplies a local byte artifact store and a literal retrieval adapter. Neither component is a vector database, a semantic ranker, a secret manager or a permission service.

LocalArtifactStore

```
from trailforge.artifacts import LocalArtifactStore

scope = ("tenant-a", "resource-a")
artifacts = LocalArtifactStore(
    "artifacts", tenant_id=scope[0], resource_id=scope[1]
)
receipt = artifacts.put(scope, b"checked result\n", "text/plain")
assert artifacts.get(scope, receipt) == b"checked result\n"
```

The default artifact byte cap is 1,048,576, configurable from one to 16,777,216. Scope must match exactly. Objects are stored below a scope-digest directory under a content-digest name; existing content is checked rather than silently trusted. The receipt binds scope digest, raw-byte content digest, size and media type, with issuer trailforge.local_artifacts/0.1.

Reads validate the closed receipt shape and current content. Local paths reject link aliases and nonregular files. There is no built-in encryption, garbage collection, aggregate disk quota or customer authentication. A per-object maximum does not prevent a host from accumulating many objects; set a host retention/storage policy.

LocalRetrieval

Construct LocalRetrieval with a directory, exact scope tuple, revision, allowlisted paths and a live_guard. Construction binds a digest of the entire root tree. An edit outside the allowlist can therefore invalidate the binding. Retrieval checks scope, revision, the live guard and the tree digest before and after reading.

Retrieval bound	Behavior
Allowed files	1-32 canonical paths.
Query	At most 256 UTF-8 bytes; case-insensitive literal substring matching.
Result limit	Default 4; range 1-8.
Read budget	At most 65,536 bytes per file and 262,144 scanned bytes per request.
Selection	Sorted allowlist order; returns whole matching file text, not ranked chunks.
Receipt	Path, revision, scope/snapshot/content digests, text, trust label and receipt_id.

Treat retrieved content as data

The receipt marks trust=UNTRUSTED_SOURCE_DATA and retrieval=EXACT_LITERAL. A correct digest means the observation matches the captured revision; it does not promote comments or instructions inside a file to host policy. Maintain that distinction when building prompts. A source comment asking an agent to upload tokens or skip checks is evidence to inspect, not an authority to obey.

Extending retrieval

A future host may supply search, embeddings or a classical ranking model through the retrieval port. Preserve exact scope/revision and source receipt binding, measure retrieval recall on relevant tasks and revalidate cited bytes at acceptance. Do not claim the existing literal implementation performs semantic search or learned ranking.

14 / Publication approvals and reconciliation

External effects need a stricter boundary than producing a candidate. GuardedPublisher binds an Intent, stores a short-lived approval and records an operation before calling the application publisher. Its supplied backend is a durable local simulator. A real GitHub, email or deployment adapter is host work.

Intent and approval identity

Intent fields are `tenant_id`, `resource_id`, `goal_digest`, `policy_digest`, `revision`, `action` and `payload`. The canonical binding also includes `payload_digest`, `adapter_identity` and `mode`. Approval therefore applies to that exact action, revision and payload under the same scope/policy; it is not a broad permission for whatever the model proposes next.

The `live_guard` receives a detached binding, actor and action and must return exactly `True`. Guard exceptions deny the action. `HOST_EXTERNAL` identity is supplied by the host authority; the local simulator uses a `LOCAL_OPERATOR_LABEL`, not an authenticated customer identity. The host must validate current membership, policy and repository revision at the actual external-write boundary.

Operation	What it does
approve	Runs APPROVE guard; creates bound approval. Default TTL 300s; allowed 1-3,600s.
revoke	Runs REVOKE guard; marks a matching approval revoked. Does not undo an accepted effect.
publish	Runs EXECUTE guard; atomically consumes a valid approval and creates a unique SENDING operation before send.
inspect_operation	Returns the exact bound operation observation, without dispatching.
reconcile	Runs RECONCILE guard; calls <code>adapter.inspect</code> , never another execute.

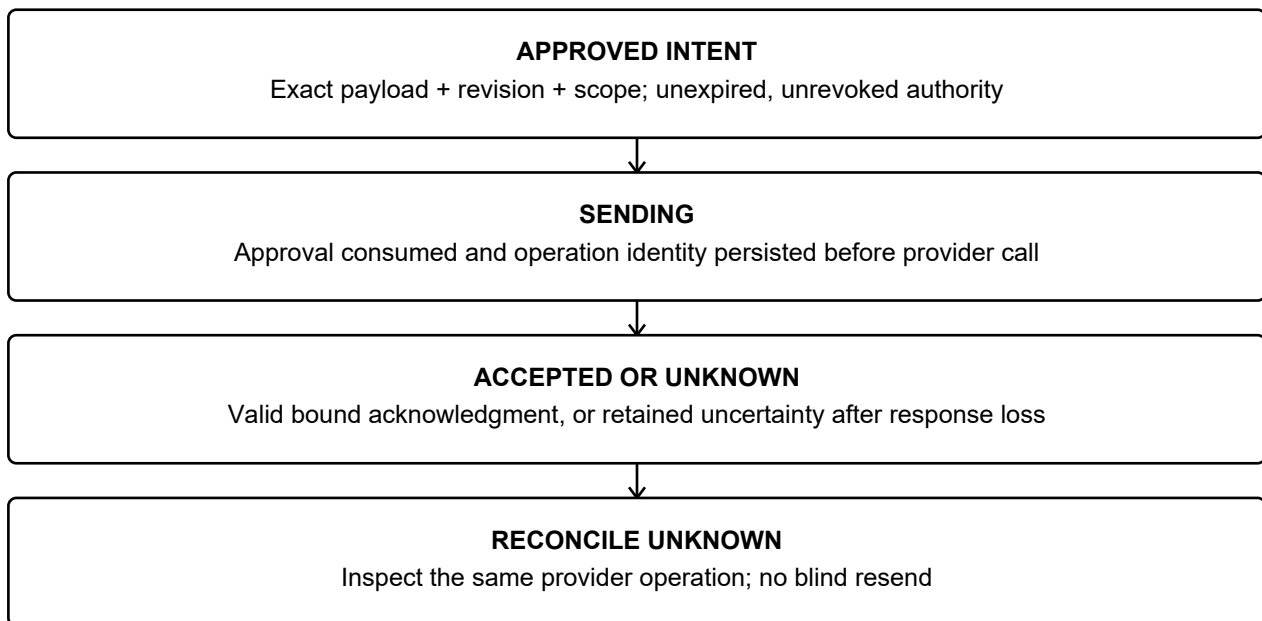


Figure 5. Durable local intent deduplication does not imply universal provider exactly-once execution.

Ambiguity is retained

The provider acknowledgment must match the operation and binding and contain the closed accepted receipt shape. A timeout, lost reply or invalid acknowledgment results in UNKNOWN. Reconciliation can establish ACCEPTED only from a valid inspected observation. It does not infer success from elapsed time, retry with a different identity or authorize a replacement payload.

An existing exact-intent operation can be returned on a repeated publish after the live guard, before checking whether the old approval is now expired, revoked or consumed. This is an observation replay; it does not execute again. Fresh dispatch still requires live authority. Independent stores do not share a universal deduplication ledger.

Adapter requirements for real providers

- Keep provider credentials out of model input and source snapshots.
- Bind a provider correlation/idempotency marker to the stable operation identity where the API permits it.
- Re-check authorization and current revision immediately before the external write, including inside the trusted adapter when required.
- Implement inspect so it can distinguish a matched accepted effect from no evidence or ambiguous evidence.
- Treat acceptance, revocation and compensation as distinct states; approval revocation cannot erase an already posted comment.

If a provider cannot inspect or deduplicate reliably, retain UNKNOWN and require an explicit host/human recovery procedure. The harness cannot manufacture an exactly-once guarantee that the provider does not offer.

Complete local publication recovery example

```
import json
import tempfile
from pathlib import Path
from trailforge.core import digest
from trailforge.publication import (
    DurableSimulationPublisher, GuardedPublisher, Intent,
)

def main():
    with tempfile.TemporaryDirectory(prefix="trailforge-publish-") as work:
        root = Path(work)
        remote = DurableSimulationPublisher(
            root / "simulation.sqlite3", lose_response=True
        )
        # Local demonstration labels only; not production authentication.
        def guard(binding, actor, action):
            return actor == "local-reviewer" and action in {
                "APPROVE", "EXECUTE", "RECONCILE", "REVOKE",
            }
        publisher = GuardedPublisher(
            root / "authority.sqlite3", tenant_id="example",
            resource_id="draft", adapter=remote, live_guard=guard,
        )
        intent = Intent(
            "example", "draft", digest({"goal": "local-demo"}),
            digest({"policy": "draft-only"}), "head-demo", "DRAFT",
            {"message": "Checked local example"},
        )
        approval = publisher.approve(intent, actor="local-reviewer")
        operation = publisher.publish(intent, approval["approval_id"])
        recovered = publisher.reconcile(operation["operation_id"], intent)
        print(json.dumps({
            "initial": operation["state"], "reconciled": recovered["state"],
        })))

if __name__ == "__main__":
    main()
```

Save as `publication_guide.py` and run with the installed package. Expected output: initial UNKNOWN, reconciled ACCEPTED. Only the independent local simulation database receives the accepted effect. The example guard validates a local label and must be replaced with real host identity/policy/revision checks for `HOST_EXTERNAL`.

15 / Legacy Review PostgreSQL authority and dispatch

The PostgreSQL adapter is for the fixed Review/controller path and its broker/outbox. It is not a drop-in backend for the neutral Harness or standalone CLI. Use a disposable test database first, then choose production authority and permissions as a host architecture decision.

Construct and migrate

```
import os
from trailforge.adapters.postgres import PostgresRunStore

store = PostgresRunStore(
    os.environ["TRAILFORGE_DATABASE_DSN"],
    tenant_id="tenant-a", resource_id="repository-a",
    schema="trailforge", lease_seconds=30,
)
store.migrate()
```

The environment variable is an application convention used in this example, not a built-in automatic configuration loader. Keep the actual DSN secret. Schema names must match `[a-z][a-z0-9_]{0,62}`. The store uses psycopg connections with a five-second connect timeout, scoped `search_path`, ten-second statement timeout and five-second lock timeout.

Migration	Authority introduced
001_foundation	Scoped runs, events, tasks, budgets and model-call reservations.
002_dispatch	Transactional <code>dispatch_outbox</code> and ready index; backfills existing nonterminal wakeups.
003_broker	Policies, principals, sessions, targets, usage, requests and audit.
004_external_sessions	Explicit external session authority support for host integration.

Migration applies under an advisory transaction lock, records checksums and rejects unexpected versions or changed applied SQL. Do not edit an already applied migration to retrofit a release. Restrict migration privilege separately from normal application operations where your deployment supports that separation.

Leases and reservations

Worker ownership uses `run_id`, `worker_id` and lease epoch. Database clock determines expiry; mutating operations require a still-live matching token. Loaded checkpoints are scoped, checksum-checked and schema-validated, and the ledger is checked against admitted mock model calls. Defaults are 100 tenant and 1,000 global declared mock units; the fixed profile still permits only two model calls per run.

Durable dispatch is a wakeup mechanism

The outbox records a stable message with the authority transition. Dispatcher claims a message, sends through the host transport and acknowledges on success. Failed delivery retains a bounded retry; attempts are capped at five with delay `min(30, 2**(attempts-1))` seconds. Expired final claims can be retired in bounded cleanup. Claim selection uses SKIP LOCKED and fenced epochs.

A transport can accept a wakeup and lose its acknowledgment; redelivery is then possible. The consumer must reload the current scoped checkpoint and acquire current authority before advancing. The notification is a hint, not permission to post a comment or reuse an obsolete approval. No managed queue service or hosted worker deployment is bundled.

Validation scope

The public alpha includes successful installed PostgreSQL tests on Linux against PostgreSQL 18.6, including schema cleanup evidence. Historical Windows PostgreSQL setup timeouts were not converted into a support claim. Windows CLI/local development support is separate from a verified Windows database deployment.

16 / Broker roles, proposals and schema catalog

The broker layer narrows Review actions to closed proposals under current role, session, scope, revision and policy authority. It is an application-controlled gate. An ActionProposal is an untrusted request until the broker validates it; model prose is never a session credential.

Role / action	Authority relationship
OPERATOR	Bootstraps local operator authority and manages permitted sessions/policy through trusted host operations.
WORKER: READ_SOURCE	Reads a permitted path under current scoped Review authority.
WORKER: MOCK_REVIEW	Invokes the fixed mock Review action; not a live provider permission.
HOST: REQUEST_SIMULATION	Requests a bound preview for the specified draft_digest.
HOST: SIMULATE_COMMENT	Executes an approved local simulation request by request_id.
REVIEWER	Decides APPROVE or REJECT on a still-current bound request.

Closed proposal shape

ActionProposal includes schema_version, run_id, binding_digest, policy_digest, expected_version, action and bounded arguments. The broker validates the action-specific argument shape, current policy, live session, role, run version, head revision and legal stage. Source-read arguments contain path; mock review has no arguments; simulation requests bind draft_digest or request_id.

Local and external sessions

LocalSession holds a secret excluded from its repr. The trusted host keeps that secret away from model context. PostgresBroker can take an explicit external_session_guard for integration with host identity. It does not implement customer GitHub OAuth, tenant provisioning or a universal identity provider.

The local-review policy declares a tool limit of 64, session duration of 3,600 seconds, pending-request duration of 900 seconds and approval duration of 600 seconds. Reviewer decisions require current request/policy authority and bounded expiry. A local comment preview has kind LOCAL_COMMENT_SIMULATION and external_effect=NONE.

Schema validation

The optional schema adapter validates the legacy closed contract catalog with Draft 2020-12 and a format checker. Unknown schema kinds are denied. Error reporting exposes field paths rather than dumping an entire sensitive payload. This is a defense against malformed structure; it does not independently verify a finding scenario or a provider effect.

The catalog contains 26 definitions: ScopeBinding, SourceReference, EvidenceReceipt, SourceArtifact, Snapshot, FindingProposal, Finding, MockOutput, ReviewReceipt, Coverage, Validation, Draft, RunCheckpoint, Task, LeaseToken, Approval, ResourcePolicy, ResourceReservation, GoalContract, AdapterDescriptor, AdapterError, DispatchMessage, ActionProposal, CommentPreview, SimulationApproval and SimulationReceipt.

Keep authority planes separate

The legacy schema Task and GoalContract are not the neutral runtime dataclasses. Likewise, a broker simulation approval is not interchangeable with a generic GuardedPublisher approval. Their protocol, payload and identity bindings are different. An application crossing those planes must explicitly translate and re-authorize the action; do not copy a PASS or APPROVE field across as authority.

17 / LangGraph facade and event telemetry

Optional orchestration and observability should expose the kernel rather than create a second source of truth. Trailforge offers a small LangGraphWorkflow facade and an allowlisted JSON event sink. Both require deliberate host composition.

LangGraphWorkflow

```
from trailforge.adapters.langgraph import LangGraphWorkflow

# harness, goal and run_id were constructed by the trusted host.
workflow = LangGraphWorkflow(harness, goal, pause_after="sum")
record = workflow.advance(run_id)
```

The facade compiles START -> kernel -> END. The kernel calls Harness.advance with the stored run identifier and supplied Goal. There is no graph checkpointing/provider, arbitrary state-update API, Command approval value or secondary durability system. Recursion limit is four. Graph state carries an observation; SQLite remains the resume authority.

The facade disables inherited automatic cloud tracing through tracing_context(enabled=False, parent=False). Hosts can export approved kernel-event projections through their own telemetry path. Calling the facade again cannot release UNKNOWN work, change the goal or grant an external approval.

JsonEventSink

```
import sys
from trailforge.telemetry import JsonEventSink
from trailforge.core import digest

event_sink = JsonEventSink(sys.stdout)
event_sink.emit({
    "schema_version": "trailforge/telemetry/0.1",
    "run_id": run_id,
    "scope_digest": digest([goal.tenant_id, goal.resource_id]),
    "kind": "TASK_RESULT_RECORDED",
    "state": record["state"],
    "units_reserved": record["units_reserved"],
})
```

This illustrative event projection requires a valid canonical UUID run_id and an allowed state. It is not a subscription to every runtime event. JsonEventSink validates exactly the six fields, writes one canonical JSON object per line, flushes and serializes writes through its instance lock.

Allowlist	Values
Kinds	ADMITTED, TASK_ADMITTED, TASK_RESULT_RECORDED, GOAL_CHECKS_EVALUATED, WAITING_HUMAN, CANCELLED, PUBLICATION_UNKNOWN, PUBLICATION_ACCEPTED.
States	READY, RUNNING, CHECKING, COMPLETED, FAILED, CANCELLED, WAITING_HUMAN, SENDING, UNKNOWN, ACCEPTED.

Allowlist	Values
Not allowed	Arbitrary prompt text, source content, tokens, actor data or added provider fields.

Host observability design

Wire emission explicitly after trusted transitions, choose durable retention and avoid placing sensitive drafts in routine logs. Keep complete authority records in the store and export minimal operational projections. Monitor unknown attempts, budget denial, publication uncertainty, dispatch exhaustion and cleanup failures. The sink supplies no automatic cloud exporter, metrics server or cross-process delivery guarantee.

18 / Worked example: a resumable data pipeline

This complete example uses no language model and no third-party SDK. It demonstrates that the harness is reusable beyond PR review: sum a bounded original dataset, pause at a checkpoint, reconstruct the host and verify the expected total.

```
import json
from trailforge.ports import (
    AdapterCapabilities, FunctionAdapter, VerifiedResult,
)
from trailforge.runtime import Goal, Harness, Task

def total(request):
    return {"total": sum(request.input["values"])}

def verify(request):
    observed = request.dependencies["sum"]["total"]
    verdict = "PASS" if observed == request.input["expected"] else "FAIL"
    return VerifiedResult(verdict, request.goal_digest,
                          {"observed": observed})

def main():
    adapters = [
        FunctionAdapter(AdapterCapabilities("example.sum/1", "TOOL"),
                        total),
        FunctionAdapter(AdapterCapabilities("example.verify/1", "VERIFY"),
                        verify),
    ]
    goal = Goal(
        "example-tenant", "data-pipeline", "Sum data and verify its total",
        (Task("sum", "example.sum/1", {"values": [1, 2, 3]}),
         Task("check", "example.verify/1", {"expected": 6}, ("sum",))),
        ("check",),
    )
    def host():
        return Harness(
            "data-pipeline.sqlite3", tenant_id=goal.tenant_id,
            resource_id=goal.resource_id, adapters=adapters,
        )
    harness = host()
    run_id = harness.start(goal)
    harness.advance(run_id, goal, pause_after="sum")
    recovered = host()
    record = recovered.advance(run_id, goal)
    print(json.dumps({
        "run_id": run_id, "state": record["state"],
        "units_reserved": record["units_reserved"],
        "result": record["tasks"]["sum"]["result"],
    }, indent=2))

if __name__ == "__main__":
    main()
```

Run and expected result

```
python data_pipeline_guide.py
# state: COMPLETED
# units_reserved: 2
# result: {"total": 6}
```

Save the code in a new working directory and run it with the installed package. The UUID differs each run. The first task consumes one unit; the verifier consumes another. The reconstructed host reuses the stored sum output instead of summing again. The check binds its observation to the Goal digest and must return a typed VerifiedResult.

Adapt the recipe safely

Replace the arithmetic callback with a bounded application tool. Keep the verifier independently tied to the intended criterion instead of simply accepting the tool output. If the tool is a remote provider, set a provider timeout and represent ambiguous outcomes through the runtime hold. If the product needs a write, do not change this task into `EXTERNAL_WRITE`; prepare a separate bound publication intent after review.

19 / Worked example: candidate creation and handoff

This complete Develop example writes sorted data to one allowed candidate file, verifies it against a separate protected fixture and resumes after BUILD. It executes trusted Python callbacks and does not run candidate-supplied code.

```
import json
import tempfile
from pathlib import Path
from trailforge.develop import CandidateWorkspace, Develop
from trailforge.ports import (
    AdapterCapabilities, FunctionAdapter, VerifiedResult,
)

def main():
    with tempfile.TemporaryDirectory(prefix="trailforge-guide-") as work:
        root = Path(work)
        candidate = CandidateWorkspace(
            root / "candidate", allowed_paths=("result.json",)
        )
        checks = root / "checks"
        checks.mkdir()
        (checks / "expected.json").write_text("[1,2,3]", encoding="utf-8")

        def build(request):
            data = json.dumps(sorted([3, 1, 2])).encode("utf-8")
            candidate.write("result.json", data)
            return {"written": "result.json"}

        def verify(request):
            expected = json.loads(
                (checks / "expected.json").read_text(encoding="utf-8")
            )
            observed = json.loads(candidate.read("result.json"))
            verdict = "PASS" if observed == expected else "FAIL"
            return VerifiedResult(verdict, request.goal_digest,
                                  {"observed": observed})

        adapters = {
            phase: FunctionAdapter(
                AdapterCapabilities("example/" + phase.lower(), "TOOL"),
                lambda request: {"task": request.task_id},
            ) for phase in ("PLAN", "RESEARCH", "DEBUG")
        }
        adapters["BUILD"] = FunctionAdapter(
            AdapterCapabilities("example/build", "TOOL", "CANDIDATE_WRITE"),
            build,
        )
        adapters["VERIFY"] = FunctionAdapter(
            AdapterCapabilities("example/verify", "VERIFY", "SCOPED_READ"),
            verify,
        )
        def host():
            return Develop(
                root / "runs.sqlite3", tenant_id="example", resource_id="data",
                objective="Write sorted data", workspace=candidate,
                checks=checks, adapters=adapters,
            )
        first = host()
        run_id = first.start()
        first.advance(run_id, pause_after="1:BUILD")
        recovered = host()
        recovered.advance(run_id)
        print(json.dumps(recovered.handoff(run_id), indent=2))

if __name__ == "__main__":
    main()
```

Run and inspect the handoff

```
python develop_guide.py
# Handoff issuer: trailforge.develop_handoff/0.1
# execution_mode: TRUSTED_HOST_CALLBACKS
# linux_isolation_verified: false
```

The temporary directory is removed when the demonstration exits. A real application must use persistent candidate/check/store locations if it needs later handoff inspection. The printed handoff is useful evidence about the checked candidate, not a deployment instruction or a claim of Linux isolation.

Protect the verifier inputs

The checks directory must not be writable by candidate code or a model-facing build tool. A callback in this example is trusted Python and can technically reach host files; Develop detects changed protected trees but is not a privilege boundary. For an untrusted test command, use DockerSandbox and have a trusted verifier interpret the resulting receipt against the original criteria.

20 / Embedding models, tools and product profiles

A production host chooses the actual intelligence and owns the surrounding product system. Trailforge supplies bounded control and integrity primitives. Its provider-neutral design is useful only when the host preserves scope, authority and acceptance through its adapters.

A real model adapter

Register a MODEL-kind adapter with an application-owned identifier. The handler receives TaskRequest, builds a prompt from bounded input and scoped dependency observations, calls the selected provider with explicit limits and returns bounded JSON. Keep credentials, identity decisions and approval tools outside the model context. Bind material prompt/model/version choices to the task or adapter identity so a resumed run is not silently a different experiment.

Do not let a provider timeout mean success. If the handler cannot establish its result or violates the output contract, the neutral runtime records uncertainty and retains its reservation. Token usage, price ceilings, cancellation and provider-side idempotency are adapter/host concerns; declared runtime units do not measure them automatically.

A real verifier

Define an objective predicate and preserve its independent inputs. A verifier might interpret a protected test run, compare a transformed dataset against invariants or validate cited source bytes and a reproduced failure. Return PASS only for that predicate; FAIL for a established criterion failure; UNKNOWN for incomplete or ambiguous evidence. Bind the returned VerifiedResult to request.goal_digest.

PR review host recipe

- Authenticate the user and repository installation outside the harness; resolve current base/head revisions under permitted access.
- Capture bounded changed files and selected context with exact scope/revision receipts.
- Invoke model/tool adapters for proposed findings; preserve source as untrusted data.
- Validate evidence and product acceptance separately. Apply required Python and JavaScript/TypeScript evaluation coverage.

- Create a draft for a reviewer. Bind approved publication to the exact current revision and payload, then use a real provider-specific publisher.

This describes how a product can embed the primitives. The released standalone harness does not include the live GitHub intake, customer sign-in, model reviewer or real comment publisher required to implement that whole recipe.

Future incident-triage profile

A future profile can bind a failed build or issue, permit only selected logs/repository files, collect revision-bound observations, propose a diagnosis and candidate fix, and require protected reproduction checks plus human review. Do not auto-apply or deploy a fix based on a diagnosis. The profile and its evaluations are future application work, not an implemented Trailforge alpha feature.

Where learned components may help

Classical ML could rank changed files, prioritize historical failure signatures or route expensive analysis when labeled data and measured gains justify it. Decision models could recommend a bounded next action. Neither should replace identity, budgets, current revision, evidence integrity or external approval. No classical model, learned decision engine or calibrated routing policy is built into this alpha. Start with deterministic rules and add learning only against a product-specific baseline.

21 / Security model and threat boundaries

The main security principle is to distinguish untrusted observations from trusted authority. A source file, model proposal, provider reply or queue body can contain useful evidence, but it cannot grant access or rewrite policy. The host, local authority storage and configured privileged infrastructure remain trusted.

Threat	Implemented boundary / remaining host duty
Prompt injection in source	Source trust labels and bounded contracts preserve provenance. Host prompts/tools must refuse source-supplied authority.
Cross-scope reuse	Exact tenant/resource/goal/revision/policy digests on relevant records. Host must authenticate who may choose that scope.
Forged or stale evidence	Exact file digest, line range, quote, revision and receipt validation. Semantic defect validity still requires review.
Concurrent/late worker	CAS, owner identity and lease fencing; no promise of preempting an already running callback.
Uncertain external write	Operation persisted before execute; UNKNOWN retained; inspect-only reconciliation. Provider correlation is host-specific.
Untrusted test execution	Linux Docker controls, immutable mounts and bounded cleanup when configured; trusted callbacks alone are not a sandbox.
Criteria mutation	Develop binds/checks protected tree and candidate continuity. Host permissions must stop unauthorized writers.
Authority-store compromise	Checksums catch inconsistency, not a privileged rewrite with recomputed hashes. Secure DB/filesystem access.
Credential/source leakage	Allowlisted telemetry and source-limited inspection; host protects draft quotes, exports and provider credentials.

Identity and permission

Tenant/resource strings are bindings, not login credentials. An actor label in a local simulator is not proof of a human identity. A production live_guard must consult the current trusted identity and resource policy, and the actual provider adapter must enforce the external-write boundary. A model must not supply a forged session secret or an actor string as a substitute.

Isolation and filesystem integrity

CandidateWorkspace, artifacts and tree capture reject link aliases and enforce bounds. These checks operate in trusted host code and are not a universal time-of-check/time-of-use defense against a privileged concurrent attacker. Mount permissions, separate worker credentials and OS/container policy are host deployment decisions. Do not mount Docker sockets, host secrets or broad writable repository roots into untrusted runners.

Security reporting and alpha limits

Report vulnerabilities privately at <https://github.com/codera647/Trailforge/security/advisories/new>. Include the version, affected boundary, minimal reproduction and impact; avoid posting secrets or customer source in a public issue. The release has conformance evidence, but no independent security certification or second-person acceptance review is claimed.

Production suitability depends on the host implementation, provider behavior, execution environment, protected criteria and operational procedures. Alpha tests verify the exercised harness boundaries; they do not prove arbitrary model-generated code is safe.

22 / Operations, recovery and observability runbook

Operate the harness as a durable authority, not a stateless request wrapper. Before restarting a worker or resolving a hold, inspect the stored contract and current bindings. The safest recovery action is the one that preserves knowledge about what may already have happened.

Observation	Operator response
Live RUNNING lease	Do not start a competing attempt. Wait or inspect the owned worker; enforce callback/provider timeouts at the host.
Expired active task	Advance can retire it to UNKNOWN. Inspect provider/candidate evidence before explicit RETRY or CANCEL.
WAITING_HUMAN / budget denial	Inspect goal, attempt counts and retained tenant/global units. Do not reset authority files to recover allowance.
Verifier UNKNOWN	Identify missing evidence; preserve criteria. Retry only through authorized host resolution with available attempts/budget.
Publication UNKNOWN	Reconcile the same operation with provider inspect. Do not resend a new operation as an automatic retry.
Checks or candidate changed	Reject handoff; create a revised goal or restore a trusted snapshot under host review.
Outbox retries exhausted	Inspect durable message and transport; use the host delivery recovery process without inventing publication authority.

Startup and shutdown

On startup, pin the package/adapters, open persistent authority, validate configured caps, verify protected-checks/image identities and confirm provider credentials through the host. Keep model-facing workers separate from authority-management credentials where the deployment needs that distinction. On shutdown, stop admission first and preserve uncertain active attempts; cancellation is not proof that a remote provider stopped.

Backups and restoration

Back up authority and related artifacts consistently. The simulator has a separate durable database: preserving only the harness publication record without provider simulation state changes what reconciliation can observe. Restored approvals may be expired, and current external revision/policy still needs live validation. A restore must not be used to make an already accepted provider effect disappear from history.

Recommended operational signals

- Counts and age of WAITING_HUMAN, UNKNOWN and active leases nearing expiry.
- Declared reservation totals and remaining goal/tenant/global capacity.
- Verifier PASS/FAIL/UNKNOWN outcomes and Review coverage gaps, separated from command success.
- Publication ambiguity and reconciliation latency; dispatch delivery exhaustion.
- Runner timeout/output-limit/workspace-change and owned-container cleanup failures.

These are host monitoring recommendations, not automatically exposed Prometheus metrics. Export allowlisted event projections, avoid full prompt/source logs and retain enough private evidence for a reviewer to reconstruct an uncertain decision.

Change management

Changing an adapter implementation under an unchanged identifier can alter behavior without changing the capability digest. Version identifiers, protect application configuration and record rollout decisions. Re-test model quality and provider semantics separately from harness conformance. Deployment should follow the product owner’s acceptance, not merely a green library test run.

23 / Testing, conformance and release evidence

The public alpha was built and validated as an independently installable package. Its conformance results demonstrate specific persistence, scope, recovery and execution boundaries. They are a release snapshot, not a blanket accuracy guarantee or a claim that every integration environment is supported.

Release validation gate	Recorded result
Portable installed suites	62 tests per supported matrix cell: Ubuntu 24.04 / Windows 2022, Python 3.11 / 3.13.15.
Optional LangGraph	2 installed cases against pinned LangGraph 1.2.14.
PostgreSQL authority	56 installed cases on Linux with PostgreSQL 18.6 and psycopg 3.3.6; owned-schema cleanup verified.
Native Linux Docker	5 boundary cases using the pinned Python image.
Suite quality gate	Nonempty suites; zero failures, errors, skips and expected failures in the accepted release reports.

Release validation gate	Recorded result
Artifact/source binding	Built wheel and sdist audited against source; public asset bytes downloaded without authentication and digest-checked.

Release identity

Tag source commit: fbefaf40fb39a1429dcc328184372214f9fd849d.

Source tree: cfe3cbc5b6317fa58d3154785d6f4538250039da.

Accepted standalone push run: <https://github.com/codera647/Trailforge/actions/runs/37814372797>. The public main merge preserves the accepted source tree. The standalone export contains 107 original source files without the private product history or local input PDFs.

Public wheel and source checksums

Release asset	SHA-256
trailforge-0.1.0a1-py3-none-any.whl	adc2b862ba4a7ad5c24d6c0a575f64f89619bb6c9433493794da75b56e9db69e
trailforge-0.1.0a1.tar.gz	395b844c8ac0841a963a530e5fba20df142f3aa5a8c12b1c3b5550c1a7c05e2c

Compare against SHA256SUMS.txt on the tagged release page. On PowerShell use `Get-FileHash -Algorithm SHA256`; on a typical Linux system use `sha256sum`. Checksums bind bytes, not a separate trusted identity signature.

Repository protection

Public main requires six exact GitHub Actions checks, an up-to-date PR route and conversation resolution; administrator enforcement is enabled and force push/deletion are blocked. The sole-maintainer configuration requires zero additional human approvals. A negative candidate failed all six checks and a non-force direct push was rejected. Version tags have active update/deletion protection.

The release assets include conformance evidence, but acceptance was not a second independent human review. Owner-run product tests, live-provider model evaluation, cloud deployment and a stable release remain separate work.

Run local checks from the tagged source

Read the tagged README, docs/conformance.md and .github/workflows/conformance.yml for exact repository commands and environment requirements. Optional database and Docker suites need real configured infrastructure; an import-only check or skipped suite cannot replace them. Use the locked development requirements and installed-package test path when reproducing release claims.

Model quality is a different evaluation

Measure finding precision, recall, unsupported coverage, evidence rejection, diagnosis correctness, cost and latency using representative labeled product tasks. The bundled evaluate command tests synthetic fixtures and explicitly makes no model-quality claim. A conformance pass says the harness followed its exercised contracts; it does not say a model's explanation or patch is correct.

24 / Troubleshooting and safe diagnostic choices

Diagnose a denial from the exact contract and boundary that rejected it. Avoid making a local demonstration “work” by removing the same binding or check a production host needs. The table maps common symptoms to the next useful observation.

Symptom	Likely cause and next step
Resume rejects the run	Changed fixture/goal, scope, revision or adapter metadata. Reconstruct the admitted contract; use a new run for revised work.
ConflictError during advance/save	Live competing owner or stale state version. Reload current authority; do not force overwrite.
WAITING_HUMAN after callback	Invalid/oversized output, exception, expired admission or uncertain verifier. Inspect retained task error and evidence.
Retry still cannot execute	Attempt or declared-unit limit reached. RETRY does not refund reservations or reset attempts.
Review outcome PARTIAL	Unsupported/truncated source, role gap or rejected proposal. Inspect coverage rather than relying on exit code 0.
Quote rejected	Revision/digest/receipt mismatch or changed line endings/range. Cite exactly the captured bytes.
Handoff rejects completed run	Candidate or checks changed after VERIFY. Re-verify a reviewed new candidate/contract.
Sandbox preflight fails	Nonlocal daemon, non-Linux mode, missing pinned image or unhealthy image inspection. Fix the boundary; no host fallback.
Sandbox cleanup not confirmed	Daemon query failed or owned container remains. Inspect the owned identity; never delete unrelated containers.
Publish returns UNKNOWN	Response loss or invalid acknowledgment. Inspect/reconcile exact operation, without another execute.
CLI approve denied	Not a COMPLETE draft, missing --simulate or inconsistent checkpoint. CLI has no real publisher mode.
Retrieval invalidated after unrelated edit	The full root tree, not just the allowlist, is bound. Capture a fresh explicit revision under host policy.
PostgreSQL schema/migration denied	Invalid identifier, unknown version or changed checksum. Preserve evidence and test migration on a disposable copy.
Optional import fails	Requested extra is not installed. Install the matching pinned extra in the environment used to run the host.

Capture useful diagnostics

Record package version, run/operation identifier, protocol, state_version, scoped digest, event kind and sanitized denial. Keep actual DSNs, session secrets, access tokens and source-bearing drafts out of public reports. To report a reproducible issue, reduce it to an original fixture or small host example, state whether the adapter is trusted local or external and identify the exact OS/backend combination.

Do not guess around uncertainty

If the host cannot observe whether a provider effect happened, keep UNKNOWN. If a protected criterion changed, do not relabel the old verification. If a reservation remains consumed, do not recreate the database to retry. These actions erase authority rather than solving the cause. Recovery must preserve the facts available to the reviewer.

25 / Adoption checklist, compatibility and roadmap

A useful production adoption starts by selecting one bounded workflow and making its acceptance observable. Implement the host boundary before adding more models or broader autonomy. The checklist below is guidance for an adopter, not an assertion that all these product capabilities are built into the alpha.

Before the first integrated run

- Pin v0.1.0a1 and verify installed bytes; decide how alpha compatibility will be managed.
- Define tenant/resource identity, exact revision inputs, permitted adapters and a finite Goal.
- Specify required independent verification predicates and protected inputs.
- Choose persistent authority/artifact paths and access/backup/retention policies.
- Set declared-unit caps plus actual provider token/time/spend limits.
- Make UNKNOWN and cancellation visible to an authorized human or host operator.

Before any external write

- Implement real authenticated actor identity and current resource/policy checks.
- Bind the exact current revision, action and payload in an Intent.
- Implement provider-specific execute and inspect with stable operation correlation.
- Test response loss, revoked/expired approval, stale revision and ambiguous acceptance.
- Keep deployment/merge/publication approval separate from a workflow PASS or Develop handoff.

Before running untrusted code

- Use a tested Linux isolation boundary and explicit pinned image maintenance.
- Keep credentials, Docker control sockets and broad writable mounts outside the runner.
- Protect criteria and verify pre/post candidate binding.
- Test timeout, output limit, workspace mutation and owned-resource cleanup under real infrastructure.

Possible future directions

A generic distributed authority backend, richer retrieval, native provider adapters, calibrated model evaluation and a dedicated incident-triage profile are possible extensions. They are not promised interfaces or current release features. Preserve the existing uncertainty/approval boundaries when proposing them, and justify learned routing or decision models with measured improvements.

Contributing

Use the standalone repository contribution instructions and exact required checks. Keep changes focused on reusable harness behavior; product-specific customer flows belong in the embedding application unless an abstraction is justified. Add meaningful tests for changed authority/recovery behavior. Explain the concrete before/after behavior, protocol compatibility and validation in a pull request.

Describe guarantees precisely: declared units, integrity receipts, workflow completion and local simulation each have narrower meaning than billing enforcement, signatures, semantic correctness and real publication.

26 / API reference extracted from the release

This appendix lists public class declarations and their public method signatures from the released Python source. It is a navigation reference, not a promise that alpha internals are stable. Underscore-prefixed methods and classes are omitted. Domain-specific semantics are explained in the preceding chapters.

Dataclass fields describe constructor inputs; method signatures are extracted without executing imports. The source links in the final chapter resolve the full implementation. For declared protocols, a signature documents an interface rather than a built-in service.

trailforge.runtime

Task

Field: task_id: str

Field: adapter_id: str

Field: input: object

Field: dependencies: tuple[str, ...] = ()

Goal

Field: tenant_id: str

Field: resource_id: str

Field: objective: str

Field: tasks: tuple[Task, ...]

Field: required_checks: tuple[str, ...]

Field: allowed_effects: tuple[str, ...] = ('PURE', 'SCOPED_READ')

Field: budget_units: int = 32

Field: max_task_attempts: int = 3

Harness

`__init__(self, path, *, tenant_id, resource_id, adapters, lease_seconds=30, clock=time.time, tenant_limit=1000, global_limit=10000)`

`start(self, goal)`

`inspect(self, run_id)`

`resource_status(self)`

`advance(self, run_id, goal, *, pause_after=None)`

`resolve_unknown(self, run_id, goal, task_id, *, disposition)`

`cancel(self, run_id, goal)`

trailforge.ports

AdapterCapabilities

Field: adapter_id: str

Field: kind: str

Field: effect: str = 'PURE'

Field: protocol: str = ADAPTER_PROTOCOL

Field: max_input_bytes: int = 65536

Field: max_output_bytes: int = 65536

Field: reservation_units: int = 1

validate(self)

TaskRequest

Field: run_id: str

Field: task_id: str

Field: scope: tuple[str, str]

Field: goal_digest: str

Field: input: Any

Field: dependencies: dict[str, Any]

VerifiedResult

Field: verdict: str

Field: goal_digest: str

Field: evidence: Any

TaskAdapter

Field: capabilities: AdapterCapabilities

execute(self, request: TaskRequest) -> Any

FunctionAdapter

__init__(self, capabilities, handler)

execute(self, request)

ArtifactPort

put(self, scope: tuple[str, str], content: bytes, media_type: str) -> dict

get(self, scope: tuple[str, str], receipt: dict) -> bytes

WorkflowPort

advance(self, run_id: str) -> dict

RetrievalPort

retrieve(self, scope: tuple[str, str], revision: str, query: str, limit: int) -> list[dict]

PublisherPort

prepare(self, intent: dict) -> dict

execute(self, operation: dict) -> dict

inspect(self, operation: dict) -> dict

TelemetryPort

emit(self, event: dict) -> None

trailforge.develop

CandidateWorkspace

__init__(self, directory, *, allowed_paths, max_file_bytes=262144)

read(self, relative)

write(self, relative, content)

Develop

```
__init__(self, store, *, tenant_id, resource_id, objective, workspace, checks, adapters, rounds=1, budget_units=32, **harness_options)
```

```
start(self)
```

```
advance(self, run_id, *, pause_after=None)
```

```
continuity(self, run_id)
```

```
handoff(self, run_id)
```

trailforge.publication**Intent**

```
Field: tenant_id: str
```

```
Field: resource_id: str
```

```
Field: goal_digest: str
```

```
Field: policy_digest: str
```

```
Field: revision: str
```

```
Field: action: str
```

```
Field: payload: object
```

GuardedPublisher

```
__init__(self, path, *, tenant_id, resource_id, adapter, live_guard, clock=time.time)
```

```
approve(self, intent, *, actor, ttl_seconds=300)
```

```
revoke(self, approval_id, intent)
```

```
inspect_operation(self, operation_id, intent)
```

```
publish(self, intent, approval_id)
```

```
reconcile(self, operation_id, intent)
```

DurableSimulationPublisher

```
__init__(self, path, *, lose_response=False)
```

```
execute(self, operation)
```

```
inspect(self, operation)
```

trailforge.artifacts**LocalArtifactStore**

```
__init__(self, directory, *, tenant_id, resource_id, max_bytes=1048576)
```

```
put(self, scope, content, media_type)
```

```
get(self, scope, receipt)
```

trailforge.store**SQLiteRunStore**

```
__init__(self, path: str | Path, *, protocol=VERSION)
```

```
connect(self) -> Iterator[sqlite3.Connection]
```

```
create(self, record: dict) -> None
```

```
load(self, run_id: str) -> dict
save(self, record: dict, expected_version: int, event: str) -> dict
save_in_transaction(self, db, record: dict, expected_version: int, event: str) -> dict
events(self, run_id: str) -> list[dict]
```

trailforge.controller

OfflineController

```
__init__(self, store: RunStore, model: ModelAdapter)
start(self, review_input: ReviewInput) -> str
initial_record(cls, review_input: ReviewInput) -> dict
advance(self, run_id: str, review_input: ReviewInput, pause_after: str | None=None) -> dict
```

trailforge.core

ScopeBinding

```
Field: tenant_id: str
Field: resource_id: str
Field: base_revision: str
Field: head_revision: str
Field: input_digest: str
Field: policy_digest: str = digest(POLICY)
Field: profile_version: str = VERSION
Field: config_digest: str = digest(CONFIG)
as_dict(self) -> dict[str, str]
binding_digest(self) -> str
```

ReviewInput

```
Field: binding: ScopeBinding
Field: sources: tuple[dict, ...]
Field: mock_output: dict
computed_input_digest(self) -> str
```

ModelAdapter

```
Field: adapter_id: str
review(self, snapshot: dict, configured_output: dict) -> dict
```

RunStore

```
create(self, record: dict) -> None
load(self, run_id: str) -> dict
save(self, record: dict, expected_version: int, event: str) -> dict
```

trailforge.telemetry

JsonEventSink

`__init__(self, stream)`

`emit(self, event)`

trailforge.broker

AdmissionDenied

`__init__(self, code='ADMISSION_DENIED')`

LocalSession

Field: secret: str = field(repr=False)

trailforge.dispatch

DispatchToken

Field: message_id: str

Field: worker_id: str

Field: epoch: int

DispatchStore

`claim_dispatch(self) -> tuple[DispatchToken, dict] | None`

`acknowledge_dispatch(self, token: DispatchToken) -> None`

`retry_dispatch(self, token: DispatchToken) -> None`

WakeupTransport

`send(self, message: dict) -> None`

Dispatcher

`__init__(self, store: DispatchStore, transport: WakeupTransport)`

`dispatch_one(self) -> bool`

trailforge.adapters.retrieval

LocalRetrieval

`__init__(self, directory, *, scope, revision, paths, live_guard)`

`retrieve(self, scope, revision, query, limit=4)`

trailforge.adapters.sandbox

SandboxLimits

Field: seconds: int = 30

Field: memory_mb: int = 256

Field: pids: int = 64

Field: output_bytes: int = 1048576

`validate(self)`

DockerSandbox

`__init__(self, image, *, executable=None, limits=SandboxLimits())`

```
command(self, candidate, checks, argv, *, name)
preflight(self)
confirm_absent(self, name)
run(self, candidate, checks, argv, *, goal_digest, candidate_digest, checks_digest)
```

trailforge.adapters.langgraph

LangGraphWorkflow

```
__init__(self, harness, goal, *, pause_after=None)
advance(self, run_id)
```

trailforge.adapters.postgres

PostgresRunStore

```
__init__(self, dsn: str, tenant_id: str, resource_id: str, *, schema='trailforge', worker_id: str |
None=None, lease_seconds=30, tenant_limit=100, global_limit=1000)
```

```
connect(self)
migrate(self)
create(self, record)
load(self, run_id)
acquire(self, run_id)
release(self, token)
renew(self, token)
worker(self, run_id)
save(self, record, expected_version, event, *, token=None)
admit_model(self, record, token)
events(self, run_id)
reservations(self, run_id)
tasks(self, run_id)
budget_usage(self, run_id)
claim_dispatch(self)
acknowledge_dispatch(self, token)
retry_dispatch(self, token)
dispatch_messages(self, run_id)
```

LeasedRunStore

```
__init__(self, parent, token)
load(self, run_id)
save(self, record, expected_version, event)
admit_model(self, record)
```

trailforge.adapters.broker

PostgresBroker

```
__init__(self, store, *, external_session_guard=None)
scope(self)
bootstrap_operator(self, actor_id)
grant_session(self, operator, actor_id, roles, *, seconds=3600)
revoke_session(self, operator, target)
set_enabled(self, operator, enabled)
context(self, session)
observe_head(self, operator, run_id, expected_epoch, head_revision)
execute(self, session, proposal, *, lease=None)
inspect_request(self, session, run_id, request_id)
decide(self, session, run_id, request_id, payload_digest, current_policy, decision, *, seconds=600)
revoke_approval(self, reviewer, run_id, request_id)
usage(self, session, run_id)
controller_store(self, session)
```

BrokeredControllerStore

```
__init__(self, broker, session, leased=None)
create(self, record)
load(self, run_id)
save(self, record, expected_version, event)
worker(self)
admit_model(self, record)
```

Selected integrity and validation functions

These public helpers complement the class interfaces. Scope/revision semantics remain those of the calling profile; shape validation alone never grants authority.

trailforge.core

```
canonical(value: Any) -> str
digest(value: Any) -> str
byte_digest(value: bytes) -> str
strict_json(raw: str) -> Any
source_path(value: Any) -> str
```

trailforge.ports

```
bounded_json(value, limit)
```

trailforge.evidence

```
capture(run_id: str, binding: dict, sources: tuple[dict, ...]) -> dict
verify_snapshot(snapshot: dict, run_id: str, binding: dict) -> None
```

```
validate_output(output: dict, snapshot: dict, run_id: str, binding: dict) -> dict
```

trailforge.fixtures

```
load_fixture(directory: str | Path) -> ReviewInput
```

trailforge.contracts

```
validate_binding(binding: dict) -> None
```

```
validate_checkpoint(record: dict) -> None
```

```
validate_transition(previous: dict, candidate: dict, event: str) -> None
```

```
assert_approval_binding(approval: dict, binding_digest: str, payload_digest: str, policy_digest: str,  
action: str, now: datetime, *, run_id: str) -> None
```

trailforge.adapters.sandbox

```
tree_digest(directory, *, max_files=512, max_bytes=16777216)
```

trailforge.adapters.schema

```
validate_record(kind: str, value) -> None
```

27 / Glossary and source references

Use these terms consistently when designing a host integration or reporting a guarantee. The references point to the released implementation and evidence, so a reader can verify the alpha behavior independently.

Term	Meaning
Goal / Task	A scoped finite acceptance contract / one ordered adapter invocation in that contract.
Scope binding	Exact identity and input/revision/policy/config digests for admitted work.
Adapter capability	Declared kind, effect and bounds; not OS-level enforcement of callback behavior.
Checkpoint / CAS	Durable current record / update accepted only at the expected state version.
Receipt	Canonical digest-bound integrity record; no signature or independent identity assurance implied.
Declared unit	Configured reservation cost; not measured tokens, dollars or automatic refunds.
Lease / fencing	Time-bound attempt ownership / denial of stale result commitment.
UNKNOWN	Outcome cannot be established safely; retain admission and use explicit recovery.
Coverage	Which sources/roles were accounted for; not calibrated model quality.
Protected checks	Host-controlled criteria/input tree kept distinct from the candidate.
Intent / approval	Exact scoped effect payload / trusted host authorization for that binding.
Reconciliation	Inspect the same operation to reduce uncertainty without blind re-execution.
Handoff	Candidate/check continuity artifact; not merge or deployment permission.

Primary project references

[1] Public repository

<https://github.com/codera647/Trailforge>

[2] Alpha release and downloadable evidence

<https://github.com/codera647/Trailforge/releases/tag/v0.1.0a1>

[3] Immutable release source tree

<https://github.com/codera647/Trailforge/tree/v0.1.0a1>

[4] Release README

<https://github.com/codera647/Trailforge/blob/v0.1.0a1/README.md>

[5] Runtime and Goal/Task implementation

<https://github.com/codera647/Trailforge/blob/v0.1.0a1/src/trailforge/runtime.py>

[6] Adapter contracts

<https://github.com/codera647/Trailforge/blob/v0.1.0a1/src/trailforge/ports.py>

[7] Develop profile

<https://github.com/codera647/Trailforge/blob/v0.1.0a1/src/trailforge/develop.py>

[8] Publication boundary

<https://github.com/codera647/Trailforge/blob/v0.1.0a1/src/trailforge/publication.py>

[9] Evidence validation

<https://github.com/codera647/Trailforge/blob/v0.1.0a1/src/trailforge/evidence.py>

[10] Docker runner

<https://github.com/codera647/Trailforge/blob/v0.1.0a1/src/trailforge/adapters/sandbox.py>

[11] PostgreSQL authority

<https://github.com/codera647/Trailforge/blob/v0.1.0a1/src/trailforge/adapters/postgres.py>

[12] Source examples

<https://github.com/codera647/Trailforge/tree/v0.1.0a1/examples>

[13] Conformance workflow

<https://github.com/codera647/Trailforge/blob/v0.1.0a1/.github/workflows/conformance.yml>

[14] Accepted standalone CI run

<https://github.com/codera647/Trailforge/actions/runs/3781437279>

[15] MIT license

<https://github.com/codera647/Trailforge/blob/v0.1.0a1/LICENSE>

[16] Private vulnerability reports

<https://github.com/codera647/Trailforge/security/advisories/new>

Guide provenance

This guide was prepared from the public-alpha implementation and checked release records. Complete guide recipes are original application examples smoke-tested against the installed release. Future host recipes and operational recommendations are labeled as integration guidance. No content from the supplied learning-book PDF is reproduced.